

WHERE TO FIND SOURCE (REFERENCING INFORMATION)	HOW RELIABLE IS THIS SOURCE?	HOW USEFUL IS THIS SOURCE?
<i>The title is.....</i> <i>The author's name is....</i> <i>The date of publication was....</i> <i>The journal / magazine / website is called.....</i> <i>The URL is.....</i> <i>For AI sources, I have included a link/transcript of the original prompt/response conversation...</i>	<i>The person writing is/is not an expert and I know this because.....</i> <i>I have compared this source with other sources in order to confirm (or question) its reliability...</i> <i>The evidence suggesting this source is fact / speculation / opinion is...</i>	<i>This source gave me information about....</i> <i>This source helped me to understand.....</i> <i>This source may have limited usefulness because...</i>
This is how a breadth traversal algorithm works. It was provided by Peter Symonds College on the behalf of the AQA exam board. Here is the URL: Algorithms T4 Graph traversal algorithms (slides 27 to 38) [1]	Given that both AQA and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided a similar algorithm: Breadth First Search or BFS for a Graph GeeksforGeeks Depth First Search or DFS for a Graph - GeeksforGeeks Therefore, I believe this source is factual. [2]	This source allows me to understand a breadth first traversal. This will allow me to verify that the maze is connected. In graph theory, if a graph is connected, you can get to any node (point) from a certain node following a path. If the maze is connected then it is solvable. This means that I can now generate a random adjacency matrix representing a maze and verify that it is solvable. The limitation of this source is it only demonstrates the reliability of this algorithm on one graph.
This is how Prim's minimum spanning tree algorithm works. It was provided by Peter Symonds College on the behalf of the OCR (MEI) exam board. Here is the URL: Minimum Spanning Trees Notes.pdf (Page 3) [3]	Given that both OCR and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided a similar algorithm: Prim's algorithm in 2 minutes Therefore, I believe this source is factual.	Once I have generated a random adjacency matrix, the maze it will represent will be unreasonably complicated. To make it more realistic, I will convert the graph into a minimum spanning tree, by removing most of the unnecessary arcs. Then I will randomly add a few arcs back to make the maze have loops (realism.) This is considered a standard maze generation

	[4]	technique.
This is how Dijkstra's algorithm works. It was provided by Peter Symonds College on the behalf of the OCR (MEI) exam board. Here is the URL: Shortest Paths Notes.pdf [5]	Given that both OCR and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided a similar algorithm: Dijkstra's Algorithm Edexcel A Level Further Maths Revision Notes 2017 Therefore, I believe this source is factual. [6]	This source allows me to understand Dijkstra's algorithm. This will allow me to make a program to solve any maze that I will generate and find the shortest path through that maze. This will be useful as a benchmark to compare the agents against. The limitation of this algorithm is it only finds the shortest path. The agents are however, completely uninformed on the structure of the maze so are unlikely to perfectly follow the shortest path (hence I will need to modify the programmed shortest path timing to account for this.)
This is the type of AI that I will be using to solve the maze. It was provided by Abid Ali Awan on Oct 27, 2022 on the behalf of datacamp. Here is the URL: An Introduction to Q-Learning: A Tutorial For Beginners DataCamp [7]	I checked Abid Ali Awan's linkedin and it turns out he is a data scientist (Abid Ali Awan - KDnuggets LinkedIn) therefore he most likely has good knowledge on AI. I checked the internet and found another source which explained the q-function in a similar manner: What is Q-Learning (back to basics) Therefore, there is a lot of evidence to suggest that this source is reliable. [8]	This source allows me to understand the Q-Function. This will be the main program that will be responsible for realistically solving the maze like a person would solve the maze. This source is limited to the Q-Function and does not look further (potentially into the Deep-Q-Learning) but this should not be significant because I doubt I will be going any further than the tabular Q-Function in this project.
Here is the documentation for NumPy. According to Wikipedia, it was released in early 2005. Here is the URL: NumPy Documentation [9]	This source is 100% reliable. This is because this source has been provided by the actual creators of the library that I will be using and so they will know their own code.	This is the library that I will be most likely using for handling the adjacency matrix containing the maze information. The documentation is very large to the point where it can be difficult to locate things that I need.
Here is the documentation for Gymnasium. It was	This source is 100% reliable. This is because this	This is the library that I will be most likely using to carry

released on 2022/09/13. Here is the URL: Gymnasium Documentation [10]	source has been provided by the actual creators of the library that I will be using and so they will know their own code.	out most of the operations regarding the AI model (training testing ect.) The documentation is very large to the point where it can be difficult to locate things that I need.
Here is the documentation for seaborn. It was released in 2014. Here is the URL: Seaborn [11]	This source is 100% reliable. This is because this source has been provided by the actual creators of the library that I will be using and so they will know their own code.	This library will help me to visualize the statistics surrounding the AI model. It is arguably the least important library in the main 4 libraries that I will be using (I could complete this project without it) but it could slightly improve the quality of my project - and help explain what is actually going on.
Here is the documentation for pygame. It was released on 28 October 2000. Here is the URL: Pygame Front Page — pygame v2.6.0 documentation [12]	This source is 100% reliable. This is because this source has been provided by the actual creators of the library that I will be using and so they will know their own code.	All the other 3 libraries were primarily to construct the program in the background. However, I am not building an adjacency matrix solver I am building a maze solver, therefore, I need to provide the examiners with a visual representation of the maze as well as the process of solving the maze. This library will allow me to do this.
This is a greater insight into graph theory. It was provided by Peter Symonds College on the behalf of the OCR (MEI) exam board. Here is the URL: 02 Graphs.pdf [13]	Given that both OCR and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided similar information: Graph Theory (Edexcel A Level Further Maths): Revision Note Therefore, I believe this source is factual. [14]	This helps me to understand graph theory better. My whole project entirely operates on this graph theory. Therefore, I believe that it will be important to go into detail on the theory and terminology behind graphs before I begin to add more complexity with the numerous algorithms that I will be adding and the AI agents. These sources are good for this basic terminology, however, do not go as far into the mathematical detail (e.g. set notation ect) as sources such as wikipedia.

This is the time and space complexity notation (Big-O). It was provided by Peter Symonds College on the behalf of the AQA exam board. Here is the URL: Algorithms T2 Big-O notation (slides 3 to 7) [15]	Given that both AQA and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided similar information: Big O Notation Tutorial - A Guide to Big O Analysis - GeeksforGeeks . Therefore, I believe this source is factual. [16]	Time and space complexity is the most well known way of being able to compare the efficiency of algorithms. Since nearly my entire project operates on algorithms, I need to be able to gauge whether I have made the correct choice of algorithms (or whether my project could be further improved on.) Therefore, I would say this source is useful. However, it doesn't specifically give me the time complexity of the algorithms I am using so the source is limited (I will need to do more research).
This is the detailed explanation of how a queue works. It was provided by Peter Symonds College on the behalf of the AQA exam board. Here is the URL: Data Structures T1 Queues [17]	Given that both AQA and Peter Symonds College are well known and established institutions, I would say that this algorithm provided is accurate. Furthermore, I checked the internet and found other sources provided similar information: Queues — Ada Computer Science Therefore, I believe this source is factual. [18]	In order to explain how a breadth-first traversal works, I must explain how a queue works (because this algorithm requires a queue.) This resource will be useful as it helps me consolidate my knowledge of queues before actually writing about it. This source is good, however, is limited by not discussing how to create a queue using a list rather than an array.
This is an insight into deep-q-learning algorithms in comparison to standard Q-Learning algorithms. It was provided by Baeldung CS. Here is the URL: Q-Learning vs. Deep Q-Learning vs. Deep Q-Network Baeldung on Computer Science [19]	Upon further research, it appears that Baeldung is written by 1 author: Eugen Paraschiv. His LinkedIn says he went to university in Romania but there is very little else to back his claims. However, I did find another resource by Jfrog which had some similarities to what Eugen wrote: A brief introduction to reinforcement learning: Deep Q-learning JFrog ML This makes me believe this source is reliable.	Earlier on in this project, I had to make a choice between making an advanced deep-q-learning AI or a standard tabular q-learning AI. These sources allow me to explain why I made the choice I made. This source is decent, however, is limited by the fact that it doesn't discuss other alternative AI models (outside of reinforcement learning.)

	[20]	
This is a complete guide to how the A* algorithm works. It was provided by Rajesh Kumar on behalf of datacamp. Here is the URL: https://www.datacamp.com/tutorial/a-star-algorithm [21]	The issue with doing further research into Rajesh Kumar is that this is the name of a well known Indian actor. Therefore, all the research I do leads me to the actor rather than the data scientist Rajesh Kumar. This makes it difficult to determine whether Rajesh is credible. However, SaveMyExams describes the A* algorithm in a similar manner: A* Algorithm (OCR A Level Computer Science): Revision Note This makes me believe this source is reliable. [22]	At the end of my Dijkstra's algorithm explanation in the research review, I need to explain why I didn't use the A* algorithm even if it is more effective. In order to do this, I need to roughly understand how the A* algorithm works. This source by Rajesh Kumar is good but is far less concise than other sources such as the one provided by SaveMyExams. This makes this source more difficult to follow.
Here is the documentation for NetworkX. It was released in April 2005. Here is the URL: Reference — NetworkX 3.5 documentation [23]	This source is 100% reliable. This is because this source has been provided by the actual creators of the library that I will be using and so they will know their own code.	This library will allow me to display discrete graphs onto the screen provided by matplotlib. This source is incredibly useful as without it, I would have no effective way of providing visualisation for the mazes that I will be creating.
Here is the documentation for Matplotlib. It was released in 2003. Here is the URL: Matplotlib 3.10.7 documentation [24]	This source is 100% reliable. This is because this source has been provided by the actual creators of the library that I will be using and so they will know their own code.	This library will allow me to draw and plot items on the screen. This source is incredibly useful and is the basis for the entire visual output for my artefact. Matplotlib allows me to translate the abstract graph data produced using networkX into the final educational visualisation.

I also used Gemini to find python libraries for visualisation of my mazes. Here is the conversation that I had with Gemini:

for my EPQ I am making an AI maze solver in python. the maze is simply a graph represented as an adjacency matrix. I need a python library (set of python libraries that) that will help me visualise discrete graphs



That sounds like a fascinating project for your EPQ! Visualizing the graph representation of your maze will be a great way to present your work.

For visualizing discrete graphs in Python, the best and most widely used combination of libraries is **NetworkX** and **Matplotlib**.

Recommended Python Libraries

Here are the libraries and how they work together:

1. NetworkX

- **Primary Use:** Creating, manipulating, and analyzing complex network structures (graphs). It's excellent for handling the mathematical representation of your maze.
- **Key Features for You:**
 - It can **directly read your adjacency matrix** and convert it into a `Graph` object.
 - It provides built-in algorithms (like breadth-first search or Dijkstra's) that you might use in your AI solver.
 - It handles the nodes (maze cells) and edges (paths between cells).

2. Matplotlib (with NetworkX's drawing functions)

- **Primary Use:** Plotting and general visualization. NetworkX uses Matplotlib's backend to draw the graph.
- **Key Features for You:**
 - It allows you to **display the nodes and edges** in a standard 2D plot.
 - You can easily customize the colors, sizes, and labels of the nodes and edges to:
 - Highlight the **start and end points** of the maze.
 - Show the **path found by your AI** solver in a different color.
 - Represent obstacles or walls if you need to.

3. Optional: Pyvis (or other interactive options)

- **Primary Use:** Creating interactive, web-based graph visualizations.
 - **Key Features for You:**
 - If you want to display your graph in a web browser where a user can **drag nodes, zoom, and pan**, Pyvis is a great option. It uses the `vis.js` library under the hood.
-

Implementation Example

Here is a quick conceptual example of how you would use NetworkX and Matplotlib:

1. Create the Graph from your Adjacency Matrix

Assuming your maze is represented by a NumPy array `adj_matrix`.

🔗 Implementation Example

Here is a quick conceptual example of how you would use NetworkX and Matplotlib:

1. Create the Graph from your Adjacency Matrix

Assuming your maze is represented by a NumPy array `adj_matrix`.

Python



```
import networkx as nx
import numpy as np

# A simple 4-node graph example (replace with your maze matrix)
adj_matrix = np.array([
    [0, 1, 0, 0],
    [1, 0, 1, 1],
    [0, 1, 0, 0],
    [0, 1, 0, 0]
])

# Create the graph object
G = nx.from_numpy_array(adj_matrix)
```

2. Visualize the Graph

Python



```
import matplotlib.pyplot as plt

# Set the layout (e.g., a simple spring layout for clarity)
# For a grid-like maze, you might need to use nx.grid_2d_graph
# or explicitly define the positions of the nodes (pos={0:(x0,y0), ...})
pos = nx.spring_layout(G)

# Draw the graph
nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=1500, edge_color='black')
```

Python



```
# Add a title and show the plot
plt.title("Adjacency Matrix Graph Visualization")
plt.show()
```

3. Highlighting the Solution Path

After your AI finds the solution (a list of nodes, e.g., `path = [0, 1, 3]`), you would calculate the edges in that path and redraw the graph, highlighting the solution edges. This is a crucial step for your EPQ presentation.

Would you like me to find a specific example of how to load your graph data structure into NetworkX, or perhaps an example of how to highlight a specific path once your AI has solved the maze?



where could I find the documentation for these libraries



That's a great question—good documentation is essential for any programming project!

You can find the official, comprehensive documentation for NetworkX and Matplotlib at the following locations:

Library Documentation Links

1. NetworkX

This is where you'll find everything related to creating, manipulating, and analyzing your graph structure. It has excellent tutorials on loading data structures like adjacency matrices.

1. NetworkX

This is where you'll find everything related to creating, manipulating, and analyzing your graph structure. It has excellent tutorials on loading data structures like adjacency matrices.

- **Official Website/Documentation:**
 - **Link:** <https://networkx.org/documentation/stable/>
 - **What to Look For:** Start with the **Tutorials** section, especially for "Creating a graph from an adjacency matrix" or "Drawing." The **Reference** section contains details on specific functions like `nx.from_numpy_array()` and the different layout options (e.g., `nx.spring_layout`).

2. Matplotlib

While NetworkX handles the plotting logic, Matplotlib's documentation will be useful for customizing the final appearance of your visualization (titles, axis labels, saving the figure, etc.).

- **Official Website/Documentation:**
 - **Link:** <https://matplotlib.org/stable/contents.html>
 - **What to Look For:** The **Usage Guide** and the **Examples** are fantastic resources. Look specifically at how to customize colors (`plt.gca().collections[0].set_edgecolor()`), add titles (`plt.title()`), and save high-quality figures (`plt.savefig()`) for your EPQ report.

3. Pyvis (Optional, for Interactive Visualization)

If you decide to use the interactive option, here is the documentation for Pyvis.

- **Official GitHub/Documentation:**
 - **Link:** <https://pyvis.readthedocs.io/en/latest/>
 - **What to Look For:** The **Tutorial** section is very straightforward for setting up a basic interactive graph in an HTML file.

As you can see, Gemini played a crucial role in helping me discover the libraries that I needed to create the UI visualisation. By conversing with the LLM longer, I was able to obtain the links to the documentation where I could conduct further research on my own. This additional research that I did was critical because the use of LLM's has some large flaws:

- Gemini can firstly provide unreliable data (hallucinations.) This is because, similar to the nature of this project, the AI generates its responses on a statistical basis rather than a factual basis. Not verifying this data especially before adding any code to my program could be detrimental to my progress and could break my program .
- Furthermore, Gemini gathers its data from large datasets based from the internet, so by directly copying from Gemini, I would be effectively copying from these sources, hence I would be plagiarising. This is a serious issue and could cause my EPQ to be rejected.

- A smaller issue is to do with indirect prompt injection. LLM's are prone to high-level attacks where an LLM like Gemini accesses malicious code written by a hacker which it then accidentally provides to users like myself. If I were to copy and paste the code given by Gemini, there is a chance I could damage my computer.